

LAB 7. XÂY DỰNG CÁC CHỨC NĂNG THÊM, SỬA, XÓA DỮ LIỆU VÀ HIỂN THỊ DANH SÁCH CÓ PHÂN TRANG

Chủ đề: Xây dựng ứng dụng CRUD hoàn chỉnh với Eloquent ORM, Phân trang (Pagination) và Quản lý Thùng rác (Soft Deletes).

MỤC TIÊU BÀI LAB

- Thành thạo việc tạo và sử dụng **Resource Controller** để quản lý vòng đời dữ liệu.
- Ứng dụng thành công **Form Request Validation** để làm sạch và kiểm tra dữ liệu đầu vào.
- Triển khai phân trang dữ liệu linh hoạt trên giao diện Blade.
- Sử dụng **Flash Session** để hiển thị thông báo trạng thái (Thành công/Lỗi) trực quan.
- Hiểu và tự tay xây dựng luồng xử lý **Soft Delete (Xóa tạm)** -> **Trash (Thùng rác)** -> **Restore (Khôi phục)** / **Force Delete (Xóa vĩnh viễn)**.

THỰC HÀNH

Tiếp tục sử dụng bảng products từ **Lab 06** (bảng đã được kích hoạt SoftDeletes và có sẵn 50 bản ghi mẫu). Chúng ta sẽ hoàn thiện các tuyến (Routes) và giao diện để người dùng có thể quản lý danh sách sản phẩm này.

BƯỚC 1: ĐỊNH NGHĨA ROUTE VÀ CẤU HÌNH PHÂN TRANG

1. Khai báo Route:

Mở routes/web.php, sử dụng Route::resource để tự động sinh ra 7 tuyến CRUD mặc định, đồng thời thêm các route xử lý Thùng rác (Trash).

```
use App\Http\Controllers\ProductController;
use Illuminate\Support\Facades\Route;

// Tuyến quản lý Thùng rác (Phải đặt TRƯỚC Route resource để tránh bị ghi đè)
Route::prefix('products')->name('products.')->group(function () {
```

```
Route::get('/trash', [ProductController::class, 'trash'])->>name('trash');

Route::post('/{id}/restore', [ProductController::class, 'restore'])->>name('restore');

Route::delete('/{id}/force-delete', [ProductController::class, 'forceDelete'])->name('forceDelete');
});

// Tuyến CRUD tự động (index, create, store, show, edit, update, destroy)
Route::resource('products', ProductController::class);
```

2. Cấu hình giao diện Phân trang (Pagination) sang chuẩn Bootstrap:

Mặc định Laravel dùng Tailwind CSS để hiển thị các nút chuyển trang. Nếu dự án đang dùng Bootstrap 5, hãy mở file `app/Providers/AppServiceProvider.php` và cập nhật hàm `boot()`:

```
namespace App\Providers;

use Illuminate\Support\ServiceProvider;
use Illuminate\Pagination\Paginator; // Import class này

class AppServiceProvider extends ServiceProvider
{
    public function boot(): void
    {
        // Ép hiệu ứng phân trang sử dụng cấu trúc HTML của Bootstrap 5
        Paginator::useBootstrapFive();
    }
}
```

BƯỚC 2: TẠO FORM REQUEST ĐỂ VALIDATE DỮ LIỆU

Để Controller được "gọn" và sạch sẽ (Clean Code), ta bóc tách tầng kiểm tra dữ liệu ra một file riêng bằng lệnh:

```
php artisan make:request StoreProductRequest
```

Mở file `app/Http/Requests/StoreProductRequest.php` vừa tạo và cấu hình luật kiểm tra:

```
namespace App\Http\Requests;
```

```
use Illuminate\Foundation\Http\FormRequest;

class StoreProductRequest extends FormRequest
{
    // Bật quyền thực thi Validation (đổi từ false thành true)
    public function authorize(): bool
    {
        return true;
    }

    // Định nghĩa các quy tắc kiểm tra (Rules)
    public function rules(): array
    {
        return [
            'name' => 'required|min:5|max:255',
            'price' => 'required|numeric|min:0',
            'stock_quantity' => 'required|integer|min:0',
            'description' => 'nullable|string',
        ];
    }

    // Tùy biến thông báo lỗi bằng tiếng Việt (Messages)
    public function messages(): array
    {
        return [
            'name.required' => 'Vui lòng nhập tên sản phẩm.',
            'name.min' => 'Tên sản phẩm phải có ít nhất 5 ký tự.',
            'price.required' => 'Vui lòng nhập giá sản phẩm.',
            'price.numeric' => 'Giá sản phẩm phải là một số hợp lệ.',
            'stock_quantity.required' => 'Vui lòng nhập số lượng tồn kho.',
        ];
    }
}
```

BƯỚC 3: TẠO RESOURCE CONTROLLER VÀ VIẾT LOGIC CRUD

Chạy lệnh tạo Controller tích hợp sẵn các hàm mẫu:

```
php artisan make:controller ProductController
```

Mở file `app/Http/Controllers/ProductController.php` và triển khai mã nguồn chi tiết cho các chức năng:

```
namespace App\Http\Controllers;

use App\Models\Product;
use App\Http\Requests\StoreProductRequest;
use Illuminate\Http\Request;

class ProductController extends Controller
{
    /**
     * 1. Hiển thị danh sách sản phẩm (Có phân trang)
     */
    public function index()
    {
        // Lấy sản phẩm mới nhất, phân trang mỗi trang 10 sản phẩm
        // Eloquent tự động bỏ qua các sản phẩm đã bị xóa tạm (Soft Deleted)
        $products = Product::latest()->paginate(10);

        return view('products.index', compact('products'));
    }

    /**
     * 2. Giao diện Form thêm mới
     */
    public function create()
    {
        return view('products.create');
    }

    /**
     * 3. Xử lý lưu sản phẩm mới vào CSDL
     */
    public function store(StoreProductRequest $request)
    {
        // $request->validated() lấy ra mảng dữ liệu đã vượt qua bộ lọc an toàn
        Product::create($request->validated());

        // Chuyển hướng về trang danh sách kèm thông báo Flash Session
        return redirect()->route('products.index')->with('success', 'Thêm
```

```
sản phẩm thành công!');  
    }  
  
    /**  
     * 4. Giao diện Form chỉnh sửa  
     */  
    public function edit(string $id)  
    {  
        $product = Product::findOrFail($id);  
        return view('products.edit', compact('product'));  
    }  
  
    /**  
     * 5. Xử lý cập nhật dữ liệu sản phẩm  
     */  
    public function update(StoreProductRequest $request, string $id)  
    {  
        $product = Product::findOrFail($id);  
        $product->update($request->validated());  
  
        return redirect()->route('products.index')->with('success', 'Cập  
nhập sản phẩm thành công!');  
    }  
  
    /**  
     * 6. Xóa tạm thời sản phẩm (Đưa vào thùng rác)  
     */  
    public function destroy(string $id)  
    {  
        $product = Product::findOrFail($id);  
        $product->delete(); // Do có SoftDeletes, hàm này sẽ nạp ngày vào  
cột deleted_at  
  
        return redirect()->route('products.index')->with('success', 'Đã  
chuyển sản phẩm vào thùng rác!');  
    }  
  
    /**  
     * 7. Hiển thị danh sách sản phẩm trong Thùng rác  
     */  
    public function trash()
```

```
{
    // onlyTrashed() chỉ lấy ra các bản ghi có deleted_at KHÁC null
    $products = Product::onlyTrashed()->latest()->paginate(10);
    return view('products.trash', compact('products'));
}

/**
 * 8. Khôi phục sản phẩm từ thùng rác
 */
public function restore(string $id)
{
    $product = Product::onlyTrashed()->findOrFail($id);
    $product->restore(); // Đặt cột deleted_at về lại null

    return redirect()->route('products.trash')->with('success', 'Khôi
phục sản phẩm thành công!');
}

/**
 * 9. Xóa vĩnh viễn sản phẩm khỏi CSDL
 */
public function forceDelete(string $id)
{
    $product = Product::onlyTrashed()->findOrFail($id);
    $product->forceDelete(); // Xóa hoàn toàn bản ghi khỏi ổ đĩa

    return redirect()->route('products.trash')->with('success', 'Đã xóa
vĩnh viễn sản phẩm khỏi hệ thống!');
}
}
```

BƯỚC 4: THIẾT KẾ CÁC GIAO DIỆN VIEW (BLADE TEMPLATE)

Để bài Lab gọn gàng, chúng ta sẽ tạo thư mục `resources/views/products/` và dùng một file layout chung hoặc viết trực tiếp tích hợp thư viện Bootstrap 5 (nhúng qua CDN).

1. Giao diện danh sách: `resources/views/products/index.blade.php`

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
<meta charset="UTF-8">

<title>Danh sách sản phẩm</title>

<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.c
ss" rel="stylesheet">
</head>
<body class="container mt-5">
    <div class="d-flex justify-content-between align-items-center mb-3">
        <h2>Quản lý sản phẩm</h2>
        <div>
            <a href="{{ route('products.create') }}" class="btn btn-
primary">Thêm sản phẩm</a>
            <a href="{{ route('products.trash') }}" class="btn btn-
warning">Thùng rác</a>
        </div>
    </div>

    @if(session('success'))
        <div class="alert alert-success">{{ session('success') }}</div>
    @endif

    <table class="table table-bordered table-striped">
        <thead>
            <tr>
                <th>ID</th>
                <th>Tên sản phẩm</th>
                <th>Giá</th>
                <th>Số lượng</th>
                <th>Hành động</th>
            </tr>
        </thead>
        <tbody>
            @foreach($products as $product)
                <tr>
                    <td>{{ $product->id }}</td>
                    <td>{{ $product->name }}</td>
                    <td>{{ number_format($product->price) }} đ</td>
                    <td>{{ $product->stock_quantity }}</td>
                    <td>
                        <a href="{{ route('products.edit', $product->id) }}"
class="btn btn-sm btn-info">Sửa</a>

```

```
                <form action="{{ route('products.destroy', $product-
>id) }}" method="POST" class="d-inline" onsubmit="return confirm('Bạn có
chắc chắn muốn xóa tạm sản phẩm này?')">
                    @csrf
                    @method('DELETE')
                    <button type="submit" class="btn btn-sm btn-
danger">Xóa</button>
                </form>
            </td>
        </tr>
    @endforeach
</tbody>
</table>

<div class="d-flex justify-content-center mt-4">
    {{ $products->links() }}
</div>
</body>
</html>
```

2. Giao diện Tạo mới / Chỉnh sửa Form (Mẫu cho file create.blade.php)

Tạo file resources/views/products/create.blade.php:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Thêm sản phẩm mới</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.c
ss" rel="stylesheet">
</head>
<body class="container mt-5" style="max-width: 600px;">
    <h2>Thêm sản phẩm mới</h2>
    <a href="{{ route('products.index') }}" class="btn btn-secondary mb-
3">Quay lại danh sách</a>

    <form action="{{ route('products.store') }}" method="POST">
        @csrf <div class="mb-3">
            <label class="form-label">Tên sản phẩm</label>
```

```
<input type="text" name="name" class="form-label form-control
@error('name') is-invalid @enderror" value="{{ old('name') }}">
    @error('name') <div class="invalid-feedback">{{ $message
}}</div> @enderror
</div>

<div class="mb-3">
    <label class="form-label">Giá bán</label>
    <input type="number" name="price" class="form-control
@error('price') is-invalid @enderror" value="{{ old('price') }}">
    @error('price') <div class="invalid-feedback">{{ $message
}}</div> @enderror
</div>

<div class="mb-3">
    <label class="form-label">Số lượng tồn kho</label>
    <input type="number" name="stock_quantity" class="form-control
@error('stock_quantity') is-invalid @enderror" value="{{
old('stock_quantity') }}">
    @error('stock_quantity') <div class="invalid-feedback">{{
$message }}</div> @enderror
</div>

<div class="mb-3">
    <label class="form-label">Mô tả sản phẩm</label>
    <textarea name="description" class="form-control" rows="3">{{
old('description') }}</textarea>
</div>

<button type="submit" class="btn btn-success">Lưu sản phẩm</button>
</form>
</body>
</html>
```

Đối với trang edit.blade.php, sinh viên chỉ cần copy giao diện trang create, đổi action trở về {{ route('products.update', \$product->id) }}, bổ sung directive @method('PUT') ngay dưới thẻ @csrf, và truyền giá trị mặc định vào các ô input bằng \$product->name, \$product->price...)

3. Giao diện Thùng rác: resources/views/products/trash.blade.php

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
  <meta charset="UTF-8">
  <title>Thùng rác sản phẩm</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.c
ss" rel="stylesheet">
</head>
<body class="container mt-5">
  <div class="d-flex justify-content-between align-items-center mb-3">
    <h2>Thùng rác sản phẩm</h2>
    <a href="{{ route('products.index') }}" class="btn btn-
secondary">Quay lại danh sách</a>
  </div>

  @if(session('success'))
    <div class="alert alert-success">{{ session('success') }}</div>
  @endif

  <table class="table table-bordered table-hover">
    <thead class="table-dark">
      <tr>
        <th>ID</th>
        <th>Tên sản phẩm</th>
        <th>Hành động</th>
      </tr>
    </thead>
    <tbody>
      @foreach($products as $product)
        <tr>
          <td>{{ $product->id }}</td>
          <td>{{ $product->name }}</td>
          <td>
            <form action="{{ route('products.restore', $product-
>id) }}" method="POST" class="d-inline">
              @csrf
              <button type="submit" class="btn btn-sm btn-
success">Khôi phục</button>
            </form>

            <form action="{{ route('products.forceDelete',
$product->id) }}" method="POST" class="d-inline" onsubmit="return
```

```
confirm('CẢNH BÁO: Hành động này sẽ xóa dữ liệu vĩnh viễn và không thể hoàn
tác!')">

        @csrf
        @method('DELETE')
        <button type="submit" class="btn btn-sm btn-
danger">Xóa vĩnh viễn</button>
    </form>
</td>
</tr>
@empty
<tr>
    <td colspan="3" class="text-center text-muted">Thùng rác
trống.</td>
</tr>
@endforelse
</tbody>
</table>

<div class="d-flex justify-content-center mt-4">
    {{ $products->links() }}
</div>
</body>
</html>
```

BƯỚC 5: CHẠY KIỂM THỬ ỨNG DỤNG

Khởi động server bằng lệnh php artisan serve, sau đó truy cập vào URL: <http://localhost:8000/products> để kiểm thử toàn bộ chu trình xử lý:

1. Xem danh sách phân trang (Click trang 1, 2, 3... dưới chân bảng).
2. Thử bấm "Xóa" một vài sản phẩm xem chúng có biến mất khỏi danh sách chính không.
3. Bấm vào nút "Thùng rác" kiểm tra xem các sản phẩm vừa xóa đã nằm ở đó chưa.
4. Thử nghiệm tính năng "Khôi phục" và "Xóa vĩnh viễn" để kiểm tra tính toàn vẹn của dữ liệu trong Database.